

Playwright Locators - Quick Notes

Beginner-friendly reference based on your practice code

1. Locator Overview

Locator	What it locates	Typical example
<code>getByRole()</code>	Elements using explicit/implicit accessibility roles and accessible names	button, checkbox, textbox
<code>getByText()</code>	Non-form elements by visible text content	heading, paragraph, div
<code>getByLabel()</code>	Form controls associated with a label	Full Name, Email Address
<code>getByPlaceholder()</code>	Input controls by placeholder text	Search Flowers...
<code>getByAltText()</code>	Usually images by their text alternative / alt attribute	Rose
<code>getByTitle()</code>	Elements by their title attribute	Tooltip/title text
<code>getByTestId()</code>	Elements using data-testid (or configured test-id attribute)	right-click-box

2. Import Statement

Use Playwright test utilities and the Locator type when you want to explicitly type a locator variable.

```
from @playwright/test: test, expect, Locator
```

3. getByAltText()

Use this when an image or another element has meaningful alternative text, usually through the alt attribute. It is useful for locating images such as logos, product images, or icons with accessible alternative text.

```
const logo: Locator = page.getByAltText("Rose"); await expect(logo).toBeVisible();
```

Tip: The text must match the element's accessible alternative text. If the image has no useful alt text, choose another locator.

4. getByText()

Use this mainly for non-interactive content when you want to find an element based on the text displayed to the user.

```
await expect(page.getByText("Providing Automation Testing Courses & Training")).toBeVisible();
```

Remember: For buttons, checkboxes, links, and other interactive controls, `getByRole()` is often more robust and accessible.

5. getByRole()

`getByRole()` is one of the preferred Playwright locator strategies. It identifies an element by its accessibility role, often together with its accessible name.

```
await page.getByRole('checkbox', { name: 'In Stock only' }).click(); await
page.getByRole('button', { name: 'Sign In / Sign Up' }).click();
```

Common roles: button, checkbox, link, textbox, heading, radio, combobox, option, img.

6. getByLabel()

Use `getByLabel()` for form controls that are associated with a visible label. This is generally cleaner than using CSS selectors or XPath for standard forms.

```
await page.getByLabel("Full Name").fill("Kushagra Chaturvedi"); await
page.getByLabel("Email Address").fill("kushagra.chaturvedi.bhargava@gmail.com");
```

Best for: inputs, checkboxes, radio buttons, and other form controls with accessible labels.

7. getByPlaceholder()

Use `getByPlaceholder()` when the input has a placeholder that clearly identifies what should be entered.

```
await page.getByPlaceholder("Search Flowers...").fill("Rose");
```

Note: A placeholder is not the same as a label. Prefer `getByLabel()` when a proper form label exists; use placeholder-based location when the placeholder is the reliable identifier.

8. getByTitle()

Use `getByTitle()` to locate an element by its title attribute.

```
await page.getByTitle("Close").click();
```

9. getByTestId()

Use `getByTestId()` when your application provides a stable test identifier such as `data-testid`. This is especially useful when visible text, roles, or labels are not stable or unique.

```
await page.getByTestId("right-click-box").click();  
Example HTML: <div data-testid="right-click-box">...</div>
```

10. setTimeout()

Your code uses `setTimeout(3000)` to pause execution for 3 seconds. It can be useful while learning/debugging, but it is usually **not recommended in real automation tests** because fixed waits make tests slower and can still be unreliable.

```
await page.setTimeout(3000);
```

Prefer: Playwright auto-waiting, expect assertions, or explicit waits for a meaningful UI condition. For example, wait for an element to be visible instead of blindly waiting 3 seconds.

11. Your Practice Flow - Clean Version

The following keeps the main locator examples from your exercise together in a cleaner form.

```
import { test, expect, Locator } from "@playwright/test"; test("Playwright Locators",  
async ({ page }) => { await page.goto("https://engineerdiaries.com/ui-framework"); const  
logo: Locator = page.getByAltText("Rose"); await expect(logo).toBeVisible(); await  
expect( page.getByText("Providing Automation Testing Courses & Training")  
).toBeVisible(); await page.getByRole("checkbox", { name: "In Stock only" }).click();  
await page.getByRole("button", { name: "Sign In / Sign Up" }).click(); await  
page.goto("https://engineerdiaries.com/contactus"); await page.getByLabel("Full  
Name").fill("Kushagra Chaturvedi"); await page.getByLabel("Email  
Address").fill("kushagra.chaturvedi.bhargava@gmail.com"); await  
page.goto("https://engineerdiaries.com/ui-framework"); await  
page.getByPlaceholder("Search Flowers...").fill("Rose"); await  
page.getByTestId("right-click-box").click(); });
```

12. Locator Selection Cheat Sheet

Situation	Prefer
Button / checkbox / link	<code>getByRole()</code>
Input with a proper label	<code>getByLabel()</code>
Input identified by placeholder	<code>getByPlaceholder()</code>

Situation	Prefer
Visible non-interactive text	<code>getByText()</code>
Image with meaningful alt text	<code>getByAltText()</code>
Element with stable title	<code>getByTitle()</code>
Element with a stable test ID	<code>getByTestId()</code>

Key takeaway: Choose locators based on how a real user or assistive technology identifies the element. In most cases, prefer role, label, text, placeholder, or other user-facing attributes before relying on brittle CSS/XPath selectors.